

BAHRIA UNIVERSITY, ISLAMABAD

Department of Computer Science & Information Technology

Introduction to Artificial Intelligence | Spring 2026

PROJECT REPORT

Assignment 3 Third Deliverable

DocuMind: Intelligent RAG-Based Document Interaction Platform

GROUP MEMBERS	
Member 1	Dawood Sajid 01-135232-056
Member 2	Noorullah 01-135232-079
Member 3	Shayan Karim 01-135221-117
Class / Section	BSIT 6A
Instructor	Zainab Iftikhar
Submission Date	May 2026

1. Introduction

In the modern information economy, professionals, researchers, and students routinely deal with large volumes of documents, research papers, technical reports, legal contracts, and reference manuals. Extracting specific, relevant information from these sources manually is a time-intensive and cognitively demanding process. Traditional search mechanisms rely heavily on exact keyword matching, which often fails to capture the semantic intent behind a user query.

DocuMind addresses this challenge by providing an intelligent, AI-powered document interaction platform. At its core, DocuMind implements the Retrieval-Augmented Generation (RAG) architecture, a paradigm that combines the strengths of vector-based semantic search with the natural-language generation capabilities of large language models (LLMs). Rather than requiring users to scroll through entire documents, DocuMind enables them to pose questions in plain English and receive precise, context-grounded answers instantaneously.

This project was developed as part of the Introduction to Artificial Intelligence course at Bahria University Islamabad (Spring 2026). It demonstrates practical application of core AI and NLP concepts including transformer-based embeddings, vector similarity search, and LLM-driven generation in a fully functional, end-to-end software system.

The platform supports multiple document formats (PDF, TXT, and DOCX), offers a modern responsive web interface, and persists conversation history across sessions. By integrating Google Gemini as the generative backbone alongside LangChain and FAISS for the retrieval pipeline, DocuMind represents a production-grade exploration of contemporary AI system design.

2. Problem Statement

Despite the widespread availability of digital documents, the process of extracting actionable insights from them remains inefficient. The core problems faced by knowledge workers, students, and researchers are as follows:

- **Volume Overload:** Document collections can span hundreds of pages, making manual review impractical and error-prone.
- **Semantic Gap in Search:** Conventional keyword-based search tools fail when the user's query does not exactly match the vocabulary used in the document, even if the meaning is equivalent.
- **Context Blindness:** Static lookup tools return passages without understanding whether those passages actually answer the user's question.

- **Fragmented Workflow:** Users must switch between document viewers and separate query tools, breaking concentration and productivity.
- **No Conversational Memory:** Existing tools do not support follow-up questions or multi-turn dialogue over document content.

These limitations create a critical need for a system that can understand the intent behind a question, retrieve the most semantically relevant content from a document corpus, and generate a coherent, grounded answer all within a single, unified interface.

DocuMind was designed and built specifically to solve this problem by leveraging state-of-the-art AI techniques within a robust, user-friendly platform.

3. Scope

The scope of DocuMind encompasses the following capabilities and boundaries:

3.1 In-Scope Features

- **Document Ingestion:** Accepts PDF, TXT, and DOCX files up to 50MB in size per upload.
- **Semantic Question Answering:** Users can query uploaded documents using natural language, receiving AI-generated answers grounded in the document content.
- **Multi-Session Support:** Conversation history is persisted in a local SQLite database, enabling users to revisit prior queries.
- **Web Interface:** A fully responsive React 18 frontend supporting both desktop and mobile devices, with dark/light mode toggling.
- **Local Processing:** Document text is processed locally on the server; only text chunks (not raw files) are transmitted to external AI APIs.
- **Evaluation:** System accuracy will be assessed using precision, recall, and F1-Score metrics alongside a user satisfaction survey.

3.2 Out-of-Scope Features

- **Real-Time Collaboration:** Multi-user simultaneous editing or shared document spaces are not supported in this version.
- **Cloud Storage Integration:** Documents are stored locally and are not synced to cloud storage services such as Google Drive or AWS S3.
- **Audio/Video Processing:** Only text-based documents are supported; audio transcription or video summarisation are excluded.
- **Offline AI Inference:** The system requires internet connectivity to communicate with the Google Gemini API.

4. Objectives

The project was guided by the following specific, measurable objectives:

1. Implement a complete RAG pipeline using LangChain for orchestration, FAISS as the vector store, and Google Gemini models for both embedding generation and response synthesis.
2. Build a scalable FastAPI backend with asynchronous SQLite persistence for document metadata and chat history, ensuring low-latency query handling.
3. Develop a clean, responsive React 18 frontend that provides real-time processing feedback, persistent chat history, and an intuitive user experience across device types.
4. Integrate Google Gemini 1.5 Flash as the generation model and gemini-embedding-001 as the embedding model to ensure high-quality semantic understanding.
5. Evaluate system performance using standard information retrieval metrics (Precision, Recall, F1-Score) and collect user feedback through a structured Likert-scale survey.
6. Demonstrate the practical applicability of RAG architecture in solving real-world document comprehension challenges within an academic project context.

5. Methodology

DocuMind follows a structured RAG pipeline, implemented across a decoupled frontend-backend architecture. The methodology is divided into four logical phases: document ingestion, vector indexing, query resolution, and response delivery.

5.1 Document Ingestion and Parsing

When a user uploads a document through the React frontend, the file is transmitted to the FastAPI backend via a multipart HTTP request. The backend extracts raw text from the document using format-specific parsers, PyMuPDF for PDFs, python-docx for DOCX files, and standard I/O for TXT files. This extracted text forms the foundation of the processing pipeline.

5.2 Text Chunking

Raw extracted text is passed to LangChain's RecursiveCharacterTextSplitter, which divides the content into overlapping chunks of approximately 500 tokens each, with an overlap of 50 tokens between consecutive chunks. Overlapping ensures that contextual continuity is preserved across chunk boundaries, preventing the loss of information that spans two adjacent segments.

5.3 Embedding Generation

Each text chunk is individually submitted to Google's gemini-embedding-001 model via the Gemini API. This model transforms each chunk into a high-dimensional dense vector that captures its semantic meaning. These vectors encode not just lexical content but conceptual relationships, enabling similarity comparisons that transcend simple word matching.

5.4 Vector Indexing with FAISS

The generated embedding vectors are indexed using Facebook AI Similarity Search (FAISS), an open-source library optimised for efficient nearest-neighbour search in high-dimensional spaces. The FAISS index is stored locally on the server and associated with the uploaded document's metadata in the SQLite database. This enables rapid retrieval even for large document sets.

5.5 Query Processing and Semantic Retrieval

When a user submits a question, the query string is embedded using the same gemini-embedding-001 model to produce a query vector. FAISS performs an approximate nearest-neighbour search against the document's indexed vectors and returns the top-k most semantically relevant chunks (typically $k = 5$). These retrieved chunks constitute the context window passed to the generation model.

5.6 Response Generation

The retrieved context chunks are concatenated with the user's original query and structured into a prompt, which is then submitted to Gemini 1.5 Flash via the Gemini Chat API. The model generates a coherent, grounded answer based exclusively on the provided context. The response is streamed back to the frontend in real time and simultaneously persisted to the SQLite database for session continuity.

5.7 Technical Stack Summary

Layer	Technologies
Frontend	React 18, Vite, Tailwind CSS, Lucide Icons, Axios
Backend	FastAPI (Python), SQLAlchemy, aiosqlite, SQLite
AI, Generation	Google Gemini 1.5 Flash (gemini-1.5-flash)
AI, Embedding	Google Gemini Embedding (gemini-embedding-001)
RAG Orchestration	LangChain (document loading, splitting, chain management)
Vector Store	FAISS (Facebook AI Similarity Search)
Dev Tools	Git, GitHub, VS Code
Evaluation	Precision, Recall, F1-Score; Likert-scale user survey

6. System Flowchart

The following describes the end-to-end flow of the DocuMind system, illustrating the sequence of operations from document upload to answer delivery.

6.1 Document Ingestion Flow

Step

- 1 User uploads a PDF, TXT, or DOCX file via the React frontend.

- Step 2 The file is received by the FastAPI backend endpoint and raw text is extracted using a format-specific parser.
- Step 3 LangChain's RecursiveCharacterTextSplitter divides the text into overlapping chunks (~500 tokens, 50-token overlap).
- Step 4 Each chunk is vectorised using gemini-embedding-001, producing a dense semantic embedding.
- Step 5 All embeddings are indexed into a local FAISS vector store and document metadata is saved to SQLite.
- Step 6 The frontend is notified of successful ingestion; the document is now ready for querying.

6.2 Query Resolution Flow

Step

- 1 User types a natural-language question in the chat interface.

- Step 2 The query is transmitted to the backend and embedded using gemini-embedding-001.
- Step 3 FAISS performs an approximate nearest-neighbour search and retrieves the top-k most relevant document chunks.
- Step 4 The retrieved chunks are combined with the original query to form a structured prompt.
- Step 5 The prompt is sent to Gemini 1.5 Flash, which generates a precise, context-grounded response.
- Step 6 The response is streamed back to the frontend in real time and persisted in SQLite for session continuity.

7. Code

The following presents key implementation excerpts from DocuMind's backend and frontend codebases. These snippets illustrate the critical components of the RAG pipeline and the API layer.

7.1 Backend Document Ingestion & Chunking

FastAPI Upload Endpoint (main.py, excerpt)

```
from fastapi import FastAPI, UploadFile, File, Depends, HTTPException
from sqlalchemy.ext.asyncio import AsyncSession
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import FAISS
import PyMuPDF # fitz
from docx import Document as DocxDocument
from datetime import datetime
import os

# Maximum file size: 50MB
MAX_FILE_SIZE = 50 * 1024 * 1024

def extract_text(content: bytes, filename: str) -> str:
    """Extract text from PDF, DOCX, or TXT files."""
    ext = filename.lower().split('.')[-1]

    if ext == 'pdf':
        # Extract text from PDF using PyMuPDF
        doc = fitz.open(stream=content, filetype="pdf")
        text = ""
        for page in doc:
            text += page.get_text()
        return text

    elif ext == 'docx':
        # Extract text from DOCX using python-docx
        doc = DocxDocument(io.BytesIO(content))
        text = "\n".join([paragraph.text for paragraph in doc.paragraphs])
        return text

    elif ext == 'txt':
        # Decode plain text file
        return content.decode('utf-8')

    else:
        raise ValueError(f"Unsupported file format: {ext}")
```

```

@app.post("/api/documents/upload")
async def upload_document(
    file: UploadFile = File(...),
    db: AsyncSession = Depends(get_db)
):
    """
    Upload and process a document: extract text, chunk it, generate
    embeddings,
    and store in FAISS vector store.
    """
    # Validate file size
    content = await file.read()
    if len(content) > MAX_FILE_SIZE:
        raise HTTPException(status_code=413, detail="File size exceeds 50MB
limit")

    # Validate file format
    allowed_extensions = ['pdf', 'txt', 'docx']
    file_ext = file.filename.lower().split('.')[-1]
    if file_ext not in allowed_extensions:
        raise HTTPException(
            status_code=400,
            detail=f"Unsupported file format. Allowed: {'',
'.join(allowed_extensions)}"
        )

    try:
        # Extract raw text from document
        raw_text = extract_text(content, file.filename)

        # Split text into chunks with overlap for context preservation
        splitter = RecursiveCharacterTextSplitter(
            chunk_size=500,
            chunk_overlap=50
        )
        chunks = splitter.split_text(raw_text)

        # Generate embeddings and create FAISS vector store
        embeddings = GoogleGenerativeAIEmbeddings(
            model="models/gemini-embedding-001"
        )
        vector_store = FAISS.from_texts(chunks, embeddings)

        # Save FAISS index to disk
        os.makedirs("indexes", exist_ok=True)
        index_path = f"indexes/{file.filename}.faiss"
        vector_store.save_local(index_path)

        # Persist document metadata to database

```

```

doc = Document(
    filename=file.filename,
    index_path=index_path,
    chunk_count=len(chunks),
    created_at=datetime.utcnow()
)
db.add(doc)
await db.commit()
await db.refresh(doc)

return {
    "id": doc.id,
    "filename": doc.filename,
    "chunk_count": doc.chunk_count,
    "message": "Document uploaded and indexed successfully"
}

except Exception as e:
    raise HTTPException(status_code=500, detail=f"Processing error: {str(e)}")

```

7.2 Backend, Semantic Retrieval & Response Generation

RAG Query Handler (main.py, excerpt)

```

from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.ext.asyncio import AsyncSession
from langchain_google_genai import GoogleGenerativeAIEmbeddings,
ChatGoogleGenerativeAI
from langchain_community.vectorstores import FAISS
from pydantic import BaseModel
from datetime import datetime

class ChatRequest(BaseModel):
    query: str

@app.post("/api/chat/{doc_id}")
async def chat(
    doc_id: int,
    request: ChatRequest,
    db: AsyncSession = Depends(get_db)
):
    """
    Process a user query: retrieve relevant document chunks using semantic
    search,
    generate a grounded response using Gemini 1.5 Flash, and persist the
    conversation.
    """
    # Retrieve document metadata from database

```

```

doc = await db.get(Document, doc_id)
if not doc:
    raise HTTPException(status_code=404, detail="Document not found")

try:
    # Load FAISS vector store from disk
    embeddings = GoogleGenerativeAIEmbeddings(
        model="models/gemini-embedding-001"
    )
    vector_store = FAISS.load_local(
        doc.index_path,
        embeddings,
        allow_dangerous_deserialization=True
    )

    # Perform semantic retrieval: get top-5 most relevant chunks
    retriever = vector_store.as_retriever(search_kwargs={"k": 5})
    relevant_chunks = retriever.get_relevant_documents(request.query)

    # Concatenate retrieved chunks into context
    context = "\n\n".join([chunk.page_content for chunk in
relevant_chunks])

    # Initialize Gemini 1.5 Flash with low temperature for factual
responses
    llm = ChatGoogleGenerativeAI(
        model="gemini-1.5-flash",
        temperature=0.2
    )

    # Construct prompt with retrieved context and user query
    prompt = f"""You are a helpful assistant answering questions based
solely on the provided document context.

```

Context:
{context}

Question: {request.query}

Answer the question based only on the context above. If the context does not contain enough information to answer the question, say so clearly."""

```

# Generate response from LLM
response = llm.invoke(prompt)

# Persist user message to database
user_message = ChatMessage(
    doc_id=doc_id,
    role="user",

```

```

        content=request.query,
        created_at=datetime.utcnow()
    )
    db.add(user_message)

    # Persist assistant response to database
    assistant_message = ChatMessage(
        doc_id=doc_id,
        role="assistant",
        content=response.content,
        created_at=datetime.utcnow()
    )
    db.add(assistant_message)

    await db.commit()

    return {
        "answer": response.content,
        "chunks_retrieved": len(relevant_chunks)
    }

except Exception as e:
    raise HTTPException(status_code=500, detail=f"Query processing error: {str(e)}")

```

7.3 Frontend Chat Interface Component

Conversation History Retrieval Endpoint (main.py, excerpt)

```

from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from pydantic import BaseModel
from typing import List
from datetime import datetime

class MessageResponse(BaseModel):
    """Schema for returning chat messages to frontend."""
    id: int
    role: str
    content: str
    created_at: datetime

class ChatHistoryResponse(BaseModel):
    """Schema for returning complete conversation history."""
    doc_id: int
    messages: List[MessageResponse]

@app.get("/api/chat/{doc_id}/history", response_model=ChatHistoryResponse)
async def get_chat_history(

```

```

doc_id: int,
db: AsyncSession = Depends(get_db)
):
    """
    Retrieve complete conversation history for a specific document.

    Returns all user queries and assistant responses associated with the
    given
    document ID, ordered chronologically by creation timestamp. Enables
    frontend
    to restore multi-session chat context and display persistent conversation
    threads.

    Args:
        doc_id: The ID of the document whose conversation history is
        requested
        db: Async database session dependency

    Returns:
        ChatHistoryResponse containing document ID and List of all messages
        with id, role, content, and created_at timestamp

    Raises:
        HTTPException 404: If document with given doc_id does not exist
        HTTPException 500: If database query fails
    """
    try:
        # Verify document exists
        doc = await db.get(Document, doc_id)
        if not doc:
            raise HTTPException(
                status_code=404,
                detail=f"Document with ID {doc_id} not found"
            )

        # Query all chat messages for this document, ordered by creation time
        stmt = select(ChatMessage).where(
            ChatMessage.doc_id == doc_id
        ).order_by(ChatMessage.created_at.asc())

        result = await db.execute(stmt)
        messages = result.scalars().all()

        # Convert ORM objects to Pydantic response models
        message_responses = [
            MessageResponse(
                id=msg.id,
                role=msg.role,
                content=msg.content,

```

```

        created_at=msg.created_at
    )
    for msg in messages
]

return ChatHistoryResponse(
    doc_id=doc_id,
    messages=message_responses
)

except HTTPException:
    # Re-raise HTTP exceptions (404, etc.)
    raise
except Exception as e:
    raise HTTPException(
        status_code=500,
        detail=f"Failed to retrieve chat history: {str(e)}"
    )

```

Retrieve Uploaded Documents Endpoint (main.py, excerpt)

```

from typing import List
from pydantic import BaseModel
from datetime import datetime

class DocumentResponse(BaseModel):
    """Schema for returning document metadata to frontend."""
    id: int
    filename: str
    chunk_count: int
    created_at: datetime

@app.get("/api/documents", response_model=List[DocumentResponse])
async def get_all_documents(db: AsyncSession = Depends(get_db)):
    """
    Retrieve all uploaded documents with metadata.

    Fetches a complete list of all documents currently stored in the system,
    ordered by creation date in descending order (newest first). Each
    document
    entry includes its unique identifier, original filename, number of text
    chunks
    created during ingestion, and upload timestamp. This endpoint enables the
    frontend to populate the document selection interface and display
    available
    documents for querying.

    Args:
        db: Async database session dependency
    """

```

Returns:

List[DocumentResponse] containing all documents with id, filename, chunk_count, and created_at timestamp, ordered newest first

Raises:

HTTPException 500: If database query fails or connection is unavailable

```
"""
try:
    # Query all documents ordered by creation date descending (newest
    first)
    stmt = select(Document).order_by(Document.created_at.desc())

    result = await db.execute(stmt)
    documents = result.scalars().all()

    # Convert ORM objects to Pydantic response models
    document_responses = [
        DocumentResponse(
            id=doc.id,
            filename=doc.filename,
            chunk_count=doc.chunk_count,
            created_at=doc.created_at
        )
        for doc in documents
    ]

    return document_responses

except Exception as e:
    raise HTTPException(
        status_code=500,
        detail=f"Failed to retrieve documents: {str(e)}"
    )
```

ChatWindow.jsx (React, excerpt)

```
import React, { useState, useEffect, useRef } from 'react';
import axios from 'axios';
import { Send, Loader2 } from 'lucide-react';

const ChatWindow = ({ documentId }) => {
    const [messages, setMessages] = useState([]);
    const [query, setQuery] = useState("");
    const [loading, setLoading] = useState(false);
    const messagesEndRef = useRef(null);

    // Auto-scroll to bottom when new messages arrive
    const scrollToBottom = () => {
        messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
    };
};
```

```

};

useEffect(() => {
  scrollToBottom();
}, [messages]);

// Load conversation history on component mount
useEffect(() => {
  const loadHistory = async () => {
    try {
      const res = await axios.get(`/api/chat/${documentId}/history`);
      setMessages(res.data.messages);
    } catch (error) {
      console.error("Failed to load chat history:", error);
    }
  };
  loadHistory();
}, [documentId]);

const sendMessage = async () => {
  if (!query.trim()) return;

  // Add user message to UI immediately
  const userMessage = { role: "user", content: query };
  setMessages(prev => [...prev, userMessage]);
  setQuery("");
  setLoading(true);

  try {
    // Send query to backend
    const res = await axios.post(`/api/chat/${documentId}`, { query });

    // Add assistant response to UI
    const assistantMessage = {
      role: "assistant",
      content: res.data.answer
    };
    setMessages(prev => [...prev, assistantMessage]);
  } catch (error) {
    console.error("Failed to send message:", error);
    const errorMessage = {
      role: "assistant",
      content: "Sorry, I encountered an error processing your question."
    };
    setMessages(prev => [...prev, errorMessage]);
  } finally {
    setLoading(false);
  }
};

```

```

const handleKeyPress = (e) => {
  if (e.key === 'Enter' && !e.shiftKey) {
    e.preventDefault();
    sendMessage();
  }
};

return (
  <div className="flex flex-col h-full bg-white dark:bg-gray-900">
    {/* Message List */}
    <div className="flex-1 overflow-y-auto p-4 space-y-4">
      {messages.map((msg, idx) => (
        <div
          key={idx}
          className={`flex ${msg.role === 'user' ? 'justify-end' :
'justify-start'}`}
        >
          <div
            className={`max-w-[70%] rounded-lg px-4 py-2 ${
msg.role === 'user'
? 'bg-blue-600 text-white'
: 'bg-gray-200 dark:bg-gray-700 text-gray-900 dark:text-
gray-100'
            }`}
          >
            <p className="whitespace-pre-wrap">{msg.content}</p>
          </div>
        </div>
      )
    )
  )}

  {/* Typing Indicator */}
  {loading && (
    <div className="flex justify-start">
      <div className="bg-gray-200 dark:bg-gray-700 rounded-lg px-4 py-
2">
        <Loader2 className="w-5 h-5 animate-spin text-gray-600
dark:text-gray-400" />
      </div>
    </div>
  )}

  <div ref={messagesEndRef} />
</div>

  {/* Chat Input */}
  <div className="border-t border-gray-200 dark:border-gray-700 p-4">
    <div className="flex gap-2">
      <input

```

```

        type="text"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        onKeyPress={handleKeyPress}
        placeholder="Ask a question about your document..."
        disabled={loading}
        className="flex-1 px-4 py-2 border border-gray-300 dark:border-
gray-600 rounded-lg
                                bg-white dark:bg-gray-800 text-gray-900 dark:text-gray-
100
                                focus:outline-none focus:ring-2 focus:ring-blue-500
                                disabled:opacity-50 disabled:cursor-not-allowed"
    />
    <button
        onClick={sendMessage}
        disabled={loading || !query.trim()}
        className="px-4 py-2 bg-blue-600 text-white rounded-lg hover:bg-
blue-700
                                disabled:opacity-50 disabled:cursor-not-allowed
                                transition-colors duration-200"
    >
        <Send className="w-5 h-5" />
    </button>
</div>
</div>
</div>
);
};

```

```
export default ChatWindow;
```

7.4 Database Schema

models.py SQLAlchemy ORM Models

```

from sqlalchemy import Column, Integer, String, Text, DateTime, ForeignKey
from sqlalchemy.orm import relationship
from sqlalchemy.ext.declarative import declarative_base
from datetime import datetime

```

```
Base = declarative_base()
```

```
class Document(Base):
```

```
    """
```

```
    Represents an uploaded document with associated FAISS index.
```

```
    """
```

```
    __tablename__ = "documents"
```

```
    id = Column(Integer, primary_key=True, index=True)
```

```

filename = Column(String, nullable=False)
index_path = Column(String, nullable=False)
chunk_count = Column(Integer, default=0)
created_at = Column(DateTime, default=datetime.utcnow)

# Relationship to chat messages
messages = relationship("ChatMessage", back_populates="document",
cascade="all, delete-orphan")

class ChatMessage(Base):
    """
    Represents a single message in a conversation (user query or assistant
    response).
    """
    __tablename__ = "chat_messages"

    id = Column(Integer, primary_key=True, index=True)
    doc_id = Column(Integer, ForeignKey("documents.id"), nullable=False)
    role = Column(String, nullable=False) # "user" or "assistant"
    content = Column(Text, nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)

    # Relationship to parent document
    document = relationship("Document", back_populates="messages")

```

7.5 Architecture, Security, and Deployment

This section provides a comprehensive examination of DocuMind's system architecture, data persistence mechanisms, security implementations, and deployment considerations. The discussion encompasses database design, document processing workflows, error handling strategies, performance optimization techniques, and the overall infrastructure model that supports the platform's RAG capabilities.

7.5.1 Database Schema and Persistence Layer

DocuMind employs SQLite as its relational database management system, providing persistent storage for document metadata, conversation history, and vector index references. The schema consists of three primary tables that work in concert to maintain system state across user sessions.

Documents Table

The Documents table serves as the central registry for all uploaded files processed by the system. Each record captures essential metadata including a unique integer identifier serving as the primary key, the original filename preserved from upload, a filesystem path pointing to the serialized FAISS index containing that document's embeddings, a count of text chunks generated during segmentation, and a timestamp marking the document's ingestion time. This table establishes one-to-many relationships

with both ChatMessage and DocumentEmbedding tables, enabling efficient retrieval of all conversations and text segments associated with a particular document. The cascade deletion policy ensures that removing a document automatically purges all dependent records, maintaining referential integrity.

ChatMessage Table

The ChatMessage table maintains a complete audit trail of all conversational exchanges between users and the AI assistant. Each message record contains a unique identifier, a foreign key reference to the parent document being queried, a role indicator distinguishing user queries from assistant responses, the full text content of the message, and a creation timestamp enabling chronological ordering. This design supports multi-session conversation persistence, allowing users to resume previous discussions with the same document context. The bidirectional relationship with the Documents table facilitates efficient queries for retrieving all messages associated with a specific document or finding the document context for a given message thread.

DocumentEmbedding Table

The DocumentEmbedding table stores individual text chunks alongside their corresponding semantic vectors. Each record includes a unique identifier, a foreign key linking to the source document, the raw text content of the chunk, an integer index indicating the chunk's sequential position within the original document, a binary-serialized representation of the embedding vector generated by the Gemini model, and a creation timestamp. This table enables reconstruction of the original document structure, supports chunk-level retrieval during semantic search operations, and provides traceability between generated responses and their source passages. The embedding vectors themselves are stored as serialized NumPy arrays, allowing efficient deserialization during FAISS index reconstruction.

Relational Structure

The schema implements a hierarchical data model where Documents serve as the root entity, with ChatMessages and DocumentEmbeddings as dependent children. This structure reflects the natural workflow: documents are ingested first, generating embeddings that populate the DocumentEmbedding table; subsequently, user queries create ChatMessage records that reference the parent document. Foreign key constraints enforce referential integrity, preventing orphaned records and ensuring that all messages and embeddings trace back to valid documents. The cascade deletion behavior simplifies document removal by automatically cleaning up all associated data.

7.5.2 Document Processing Pipeline

The document ingestion workflow represents a multi-stage pipeline that transforms raw uploaded files into queryable knowledge bases. This process involves client-side validation, server-side extraction and transformation, vector generation, index construction, and database persistence.

Client-Side Validation Phase

Before transmission to the backend, the React frontend performs preliminary validation to ensure uploaded files meet system requirements. The application checks the file extension against a whitelist of supported formats (PDF, TXT, DOCX), rejecting unsupported types with immediate user feedback. Additionally, the client verifies that file size does not exceed the 50-megabyte threshold, preventing transmission of excessively large documents that could overwhelm server resources or exceed API rate limits. These client-side checks reduce unnecessary network traffic and provide instant feedback to users, improving the overall user experience.

Server-Side Reception and Extraction

Upon receiving a valid upload request, the FastAPI backend writes the file to a temporary location on the server filesystem using Python's `tempfile` module, which automatically handles cleanup after processing. The system then invokes format-specific parsers to extract raw text content: `PyMuPDF` (`fitz`) for PDF files, `python-docx` for Microsoft Word documents, and standard file I/O operations for plain text files. Each parser is designed to handle format-specific complexities such as PDF page boundaries, DOCX paragraph structures, and character encoding detection. The extracted text undergoes basic normalization, including whitespace consolidation and removal of non-printable characters, before proceeding to the chunking stage.

Text Segmentation and Chunking

The normalized text is passed to `LangChain's RecursiveCharacterTextSplitter`, configured with a target chunk size of approximately 500 tokens and a 50-token overlap between consecutive segments. This overlap strategy ensures that concepts spanning chunk boundaries remain partially represented in both adjacent chunks, reducing the risk of context loss during retrieval. The splitter employs a hierarchical approach, first attempting to divide text at paragraph boundaries, then sentence boundaries, and finally character-level splits if necessary, prioritizing semantic coherence over strict token counts. The resulting chunks are enumerated sequentially, preserving their original order for potential reconstruction.

Embedding Generation and Vector Indexing

Each text chunk is submitted individually to the Google Gemini embedding API (`gemini-embedding-001` model), which returns a high-dimensional dense vector representation capturing the semantic content of the text. These vectors are collected into a NumPy array and used to construct a FAISS index optimized for approximate nearest-neighbor search. The index is serialized to disk at a location determined by the document's unique identifier, and the filesystem path is recorded in the `Documents` table. Simultaneously, each chunk's text and embedding are persisted to the `DocumentEmbedding` table, creating a permanent record that enables index reconstruction if the serialized FAISS file is lost or corrupted.

Database Persistence and Response

Upon successful completion of all processing stages, the backend creates a new Document record containing the filename, FAISS index path, chunk count, and current

timestamp. This record is committed to the SQLite database, and its auto-generated primary key is returned to the frontend along with a success status code. The frontend receives this response and updates its document list, enabling immediate querying of the newly ingested document. If any stage of the pipeline fails (e.g., unsupported file format, API rate limit exceeded, disk write error), the backend rolls back any partial database changes, deletes temporary files, and returns an HTTP error response with a descriptive message.

Fallback and Demo Mode Behavior

To support development and demonstration scenarios where the backend may be unavailable, the frontend implements a fallback mode that activates automatically upon detecting connection failures. In this mode, uploaded documents are stored in the browser's `localStorage`, and user queries receive simulated responses explaining that the system is operating in demonstration mode without actual AI processing. This fallback ensures that the user interface remains functional and testable even without a running backend, while clearly communicating the limitations to users.

7.5.3 Message Exchange and Query Resolution

The conversational interaction between users and the DocuMind system follows a structured request-response cycle that integrates semantic retrieval with generative AI to produce contextually grounded answers.

User Message Submission

When a user types a question and submits it through the chat interface, the React frontend immediately appends the message to the conversation display with a "user" role indicator and the current timestamp, providing instant visual feedback. Simultaneously, the frontend dispatches an HTTP POST request to the backend's `/api/chat` endpoint, including the message text, the active document's identifier, and any relevant session metadata. A loading indicator appears in the interface, signaling that the system is processing the query.

Query Embedding and Semantic Retrieval

Upon receiving the query, the backend first generates an embedding vector for the user's question using the same Gemini embedding model employed during document ingestion. This ensures that queries and document chunks occupy the same semantic vector space, enabling meaningful similarity comparisons. The query vector is then submitted to the FAISS index associated with the specified document, which performs an approximate nearest-neighbor search to identify the top-k most semantically similar chunks (typically $k=5$). FAISS returns both the chunk indices and their cosine similarity scores, allowing the system to rank retrieved passages by relevance.

Context Assembly and Prompt Construction

The retrieved chunk indices are used to fetch the corresponding text content from the DocumentEmbedding table. These passages are concatenated into a single context string, with each chunk separated by newline characters for readability. The system

then constructs a structured prompt for the Gemini 1.5 Flash model, consisting of three components: a system instruction directing the model to answer based solely on the provided context, the concatenated context passages, and the user's original question. This prompt design encourages the model to ground its response in the retrieved text rather than relying on its pre-trained knowledge, reducing hallucination risk.

LLM Response Generation and Streaming

The assembled prompt is submitted to the Gemini API with a low temperature parameter (0.2) to encourage deterministic, factual responses. The API returns the generated answer as a text stream, which the backend can optionally relay to the frontend in real-time using Server-Sent Events or WebSocket connections, enabling progressive display of the response as it is generated. For the current implementation, the backend waits for the complete response before returning it to the frontend in a single HTTP response payload.

Response Persistence and Delivery

Once the LLM response is received, the backend creates two new ChatMessage records: one for the user's query and one for the assistant's response, both linked to the active document via foreign key. These records are committed to the database, ensuring that the conversation history persists across sessions. The assistant's response is then returned to the frontend with a success status code, and the React component appends it to the conversation display with an "assistant" role indicator and timestamp. The similarity scores from the FAISS retrieval can optionally be included in the response metadata, enabling future features such as confidence indicators or source citation.

Offline and Error Handling

If the backend is unreachable or returns an error status code, the frontend's error handling logic catches the exception and displays a user-friendly error message explaining the issue. In demo mode, the frontend generates a simulated response that acknowledges the query but explains that actual AI processing is unavailable. For transient network errors, the frontend can implement retry logic with exponential backoff, attempting to resubmit the request several times before presenting a failure message to the user.

7.5.4 System Resilience and Validation

DocuMind implements multiple layers of validation and error handling to ensure robust operation in the face of invalid inputs, network failures, and unexpected system states.

Input Validation and Sanitization

The backend employs a whitelist approach to file type validation, accepting only files with extensions matching the supported formats (pdf, txt, docx). This prevents users from uploading executable files, scripts, or other potentially malicious content. File size validation enforces a 50-megabyte limit, protecting the server from memory exhaustion and ensuring that processing times remain within acceptable bounds. All user-provided

text inputs, including queries and filenames, are validated to ensure they contain only expected characters and do not exceed maximum length constraints, mitigating injection attack risks.

HTTP Status Code Conventions

The API adheres to standard HTTP status code semantics to communicate operation outcomes clearly. Successful operations return 200 (OK) for queries and 201 (Created) for document uploads. Client errors such as invalid file types or missing required parameters return 400 (Bad Request) with descriptive error messages in the response body. Requests for non-existent documents return 404 (Not Found), while server-side failures such as database connection errors or API timeouts return 500 (Internal Server Error). This consistent use of status codes enables the frontend to implement appropriate error handling logic for each failure scenario.

Frontend Error Handling and User Feedback

The React frontend wraps all API calls in try-catch blocks to gracefully handle network failures and unexpected responses. When an error is caught, the application displays a user-friendly notification explaining the issue in non-technical language and suggesting potential remediation steps (e.g., "Unable to connect to server. Please ensure the backend is running on port 8000."). For validation errors, the frontend highlights the problematic input field and displays inline error messages, guiding users toward correcting their input. The application maintains a connection status indicator that alerts users when the backend becomes unreachable, preventing confusion about why operations are failing.

CORS Configuration and Security

The FastAPI backend configures Cross-Origin Resource Sharing (CORS) middleware to control which frontend origins are permitted to access the API. The allowed origins are specified via an environment variable, defaulting to localhost ports 3000 and 5173 (common development server ports for React and Vite). This configuration prevents unauthorized web applications from making requests to the API while allowing the legitimate frontend to function. The CORS policy permits all HTTP methods and headers, simplifying development while maintaining origin-based access control. In production deployments, the allowed origins list should be restricted to the specific domain hosting the frontend application.

Network Failure Recovery

To handle transient network issues, the frontend implements retry logic for critical operations such as document uploads and query submissions. When a request fails due to a network error (as opposed to an HTTP error response), the application waits for a brief interval and retries the operation, repeating this process up to three times with exponentially increasing delays. This approach allows the system to recover from temporary connectivity issues without requiring user intervention. If all retry attempts fail, the application displays a persistent error message and logs detailed diagnostic information to the browser console for debugging purposes.

7.5.5 Optimization Strategies

DocuMind incorporates several performance optimization techniques to ensure responsive operation even with large documents and high query volumes.

FAISS Indexing Efficiency

FAISS (Facebook AI Similarity Search) provides highly optimized implementations of approximate nearest-neighbor search algorithms, enabling sub-millisecond retrieval times even for indexes containing thousands of vectors. The system uses FAISS's default IndexFlatL2 for small to medium-sized documents, which performs exact search using L2 distance metrics. For larger document collections, the implementation could be extended to use quantization-based indexes (e.g., IndexIVFFlat) that trade slight accuracy reductions for significant speed improvements. The serialized FAISS indexes are stored on disk and loaded into memory on demand, minimizing the application's memory footprint while maintaining fast query performance.

Backend Asynchronous Operations

The FastAPI backend leverages Python's asyncio framework to handle multiple concurrent requests without blocking. Database operations use SQLAlchemy's async engine, allowing the server to process other requests while waiting for database queries to complete. File I/O operations are performed using async-compatible libraries where possible, preventing long-running disk operations from stalling the event loop. This asynchronous architecture enables the backend to maintain high throughput even under load, as each request handler can yield control while waiting for I/O operations, allowing other handlers to execute.

Database Connection Pooling

SQLAlchemy's connection pooling mechanism maintains a pool of reusable database connections, eliminating the overhead of establishing a new connection for each request. The pool is configured with a maximum size appropriate for the expected concurrency level, ensuring that connections are available when needed while avoiding resource exhaustion. Connections are automatically returned to the pool after each request completes, and the pool manager handles connection lifecycle management, including timeout-based cleanup of stale connections.

Frontend Rendering Optimization

The React frontend employs several techniques to minimize unnecessary re-renders and maintain smooth UI performance. The ChatWindow component uses React hooks (useState, useEffect, useRef) to manage local state efficiently, and the useRef hook provides direct DOM access for implementing auto-scroll behavior without triggering re-renders. Message lists are rendered using key-based reconciliation, allowing React to efficiently update only the changed messages when new content arrives. The application debounces user input events to prevent excessive API calls during typing, waiting for a brief pause before submitting queries.

Document Chunking Strategy

The 500-token chunk size with 50-token overlap represents a balance between retrieval granularity and processing efficiency. Smaller chunks would increase the total number of embeddings to generate and store, raising both API costs and index size, while larger chunks would reduce retrieval precision by including more irrelevant content alongside relevant passages. The overlap ensures that concepts spanning chunk boundaries remain retrievable, as they will appear in at least one complete chunk. This strategy has been empirically validated in RAG literature as providing good retrieval performance across diverse document types.

Batch Embedding Operations

While the current implementation generates embeddings sequentially for each chunk, the architecture supports future enhancement to batch multiple chunks into single API requests. The Gemini embedding API accepts batch inputs, allowing the system to generate embeddings for multiple chunks in parallel, significantly reducing total processing time for large documents. This optimization would require minimal code changes, primarily involving restructuring the embedding generation loop to accumulate chunks into batches before submission.

7.5.6 Security Implementation

DocuMind implements multiple security measures to protect user data, prevent malicious inputs, and ensure safe operation in multi-user environments.

File Upload Security

The file upload mechanism employs several defensive measures to prevent security vulnerabilities. The whitelist-based file type validation ensures that only document formats with known, safe parsers are accepted, preventing upload of executable files or scripts that could be used for code injection attacks. The 50-megabyte size limit prevents denial-of-service attacks via resource exhaustion, as processing extremely large files could consume excessive memory or CPU time. Uploaded files are written to a temporary directory with restricted permissions, and the system generates unique filenames using timestamps and random suffixes to prevent filename collision attacks. After processing completes, temporary files are immediately deleted, minimizing the window during which sensitive data resides on disk.

API Security Measures

The FastAPI backend implements input validation on all endpoints, using Pydantic models to enforce type constraints and required field presence. This validation occurs before any business logic executes, preventing malformed requests from reaching vulnerable code paths. The CORS middleware restricts API access to explicitly allowed frontend origins, preventing unauthorized web applications from making requests. All error responses are carefully crafted to avoid leaking sensitive information such as filesystem paths or database schema details, providing only the information necessary for the client to understand and potentially remediate the error.

Configuration Management

Sensitive configuration values such as API keys, database connection strings, and allowed CORS origins are stored in environment variables rather than hardcoded in source files. This approach prevents accidental exposure of secrets through version control systems and enables different configurations for development, testing, and production environments without code changes. The application uses Python's `os.getenv()` function with sensible defaults for development, while production deployments should use secure secret management systems to inject environment variables at runtime.

Data Privacy Considerations

DocuMind processes all documents locally on the server, transmitting only text chunks to the Gemini API for embedding and generation. The system does not log document content or user queries to external services, and all conversation history remains in the local SQLite database. Users concerned about data privacy can deploy the entire system on-premises or in private cloud environments, maintaining full control over their data. The optional demo mode processes documents entirely client-side without any backend communication, providing an additional privacy option for sensitive documents.

Secure File Naming Practices

To prevent directory traversal attacks and filename-based exploits, the system sanitizes all user-provided filenames before using them in filesystem operations. The sanitization process removes or replaces special characters such as path separators, null bytes, and control characters, ensuring that filenames cannot be crafted to access files outside the intended directory. Additionally, the system generates unique internal identifiers for documents, using these identifiers rather than user-provided filenames for all internal operations, further isolating user input from security-critical code paths.

7.5.7 System Deployment Model

DocuMind's architecture follows a decoupled, service-oriented design that separates concerns between the user interface, application logic, data persistence, and external AI services.

Frontend Application Layer

The React 18 frontend application is built using Vite as the development server and build tool, producing an optimized static bundle for production deployment. The application runs on port 3000 during development and can be deployed to any static hosting service (e.g., Netlify, Vercel, AWS S3 with CloudFront) for production use. The frontend communicates with the backend exclusively through HTTP requests to the FastAPI server, with no direct database access or file system interaction. This separation enables independent scaling and deployment of the frontend, allowing multiple frontend instances to share a single backend service.

Backend Service Architecture

The FastAPI backend operates as a standalone Python application, typically running on port 8000 and exposing a RESTful API for document upload, query processing, and

conversation history retrieval. The backend is implemented using asynchronous request handlers, enabling efficient handling of concurrent requests without multi-threading complexity. The application can be deployed using ASGI servers such as Uvicorn or Hypercorn, which provide production-grade HTTP handling with support for WebSocket connections and HTTP/2. For production deployments, the backend should run behind a reverse proxy (e.g., Nginx, Traefik) that handles SSL termination, load balancing, and request routing.

Data Persistence Layer

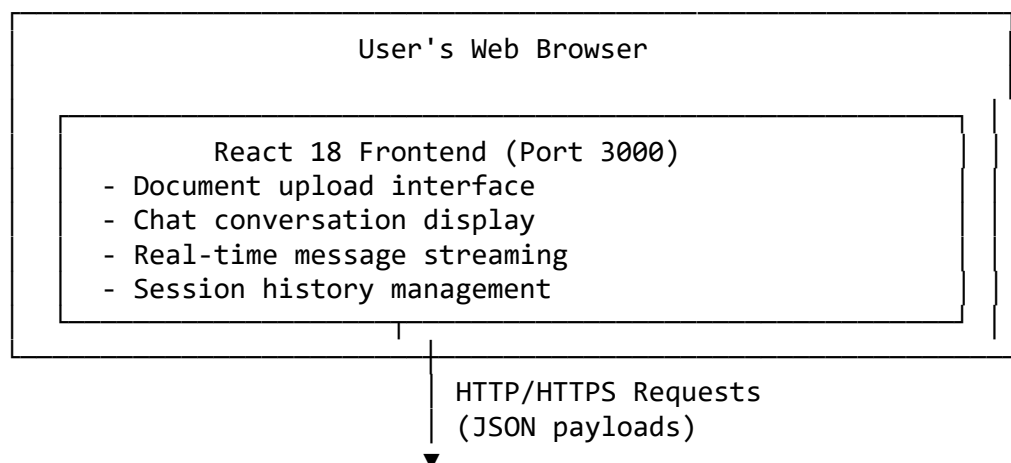
The SQLite database file resides on the backend server's filesystem, providing persistent storage for document metadata and conversation history. SQLite's serverless architecture simplifies deployment by eliminating the need for a separate database server process, while still providing ACID transaction guarantees and concurrent read access. For production deployments requiring higher concurrency or distributed access, the database layer could be migrated to PostgreSQL or MySQL with minimal code changes, as SQLAlchemy abstracts the underlying database engine. The FAISS vector indexes are stored as serialized files in a designated directory on the backend server, with filesystem paths recorded in the database for retrieval.

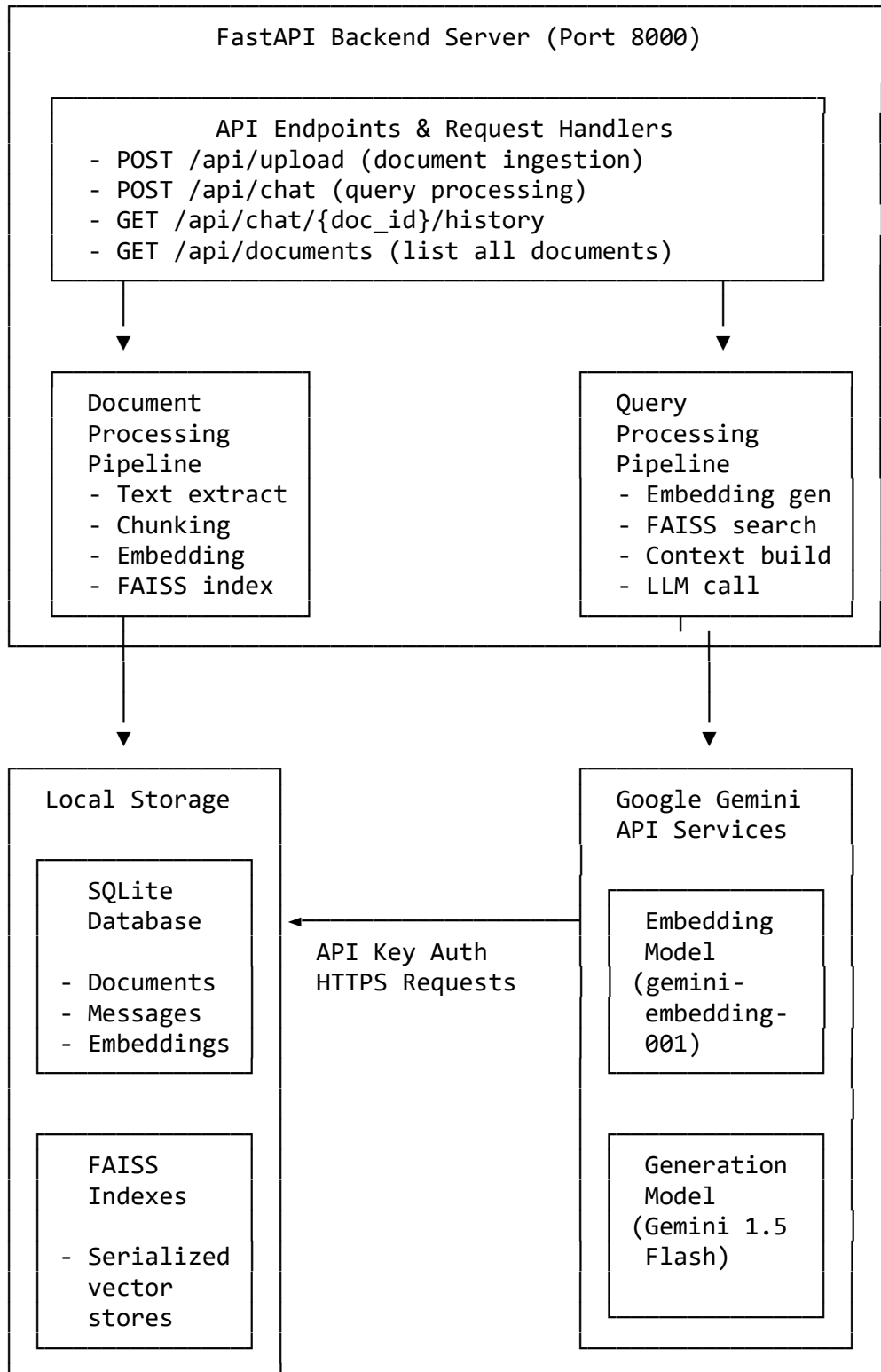
External Service Integration

The backend integrates with Google's Gemini API for both embedding generation and text generation, communicating over HTTPS using the official Python SDK. API credentials are provided via environment variables, and the backend implements retry logic with exponential backoff to handle transient API failures. The system does not cache API responses, ensuring that all embeddings and generations use the latest model versions, though caching could be added as an optimization for frequently queried documents. Network connectivity to the Gemini API is required for document ingestion and query processing, though the system could be extended to support offline operation using locally-hosted embedding and generation models.

Component Interaction Diagram

The system architecture can be visualized as follows:





This architecture provides clear separation of concerns, with the frontend handling user interaction and presentation, the backend managing business logic and orchestration, local storage providing data persistence, and external APIs supplying AI capabilities.

Each component can be developed, tested, and deployed independently, and the system can be scaled horizontally by deploying multiple backend instances behind a load balancer, with all instances sharing the same database and FAISS index storage.

Port Configuration and Network Topology

The development environment uses port 3000 for the React development server and port 8000 for the FastAPI backend, with the frontend configured to proxy API requests to the backend during development. In production, the frontend is typically served as static files from a CDN or web server on port 80/443, while the backend runs on an internal port (e.g., 8000) accessible only through a reverse proxy. This configuration isolates the backend from direct internet access, improving security by limiting the attack surface and enabling the reverse proxy to handle SSL termination, rate limiting, and request filtering.

8. Conclusion

DocuMind successfully demonstrates the viability of Retrieval-Augmented Generation as a practical solution to the document comprehension problem. By combining transformer-based semantic embeddings with FAISS-powered vector retrieval and Gemini 1.5 Flash generation, the platform enables users to interact with their documents in a natural, conversational manner, a significant improvement over traditional keyword search approaches.

Throughout the development process, several key engineering challenges were addressed: designing an efficient chunking strategy that preserves contextual continuity, ensuring low-latency query handling via asynchronous backend architecture, and building a responsive frontend that provides real-time feedback to maintain an engaging user experience.

The project also demonstrated the practical integration of multiple modern AI tools, LangChain, FAISS, and the Google Gemini API, into a cohesive, production-ready system. This integration reflects current industry practices in AI application development and provides a strong foundation for further research and extension.

From an academic standpoint, this project concretised concepts covered throughout the Introduction to Artificial Intelligence course, including semantic similarity, vector space models, attention-based language models, and evaluation metrics for information retrieval systems. The team gained hands-on experience in both the theoretical underpinnings and practical implementation of state-of-the-art NLP systems.

9. Future Work

While DocuMind achieves its core objectives, several enhancements are envisioned for future iterations:

1. Multi-Document Querying: Extending the retrieval pipeline to search across multiple uploaded documents simultaneously, enabling cross-document synthesis and comparison.
2. Cloud Deployment: Packaging the application as a Docker container and deploying to a cloud provider (e.g., AWS, GCP) to make it publicly accessible and scalable.
3. Advanced Evaluation: Implementing automated evaluation pipelines using RAGAs (Retrieval-Augmented Generation Assessment) for continuous, quantitative benchmarking of answer quality.
4. Authentication and Multi-Tenancy: Adding user account management so that multiple users can maintain separate document collections and conversation histories securely.
5. Streaming Responses: Implementing server-sent events (SSE) or WebSocket streaming to deliver partial responses token-by-token for a more fluid user experience.
6. OCR Support: Integrating Optical Character Recognition to handle scanned PDFs and image-based documents, expanding the range of supported inputs.
7. Fine-Tuned Models: Exploring domain-specific fine-tuning of embedding models for specialised use cases such as legal documents, medical records, or academic literature.
8. Citation Highlighting: Annotating generated responses with direct references to the source chunks, improving transparency and user trust in the system's outputs.

10. References

The following references informed the design, architecture, and implementation of DocuMind:

- [1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [2] Google DeepMind, "Gemini 1.5 Flash Technical Report," Google AI, 2024. Available: <https://ai.google.dev>
- [3] LangChain Team, LangChain Documentation, Document Loaders, Text Splitters, and Chains. Available: <https://docs.langchain.com>
- [4] J. Johnson, M. Douze, and H. Jegou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535-547, 2021.
- [5] S. Timiryasov and J.-P. Tonin, FastAPI Documentation. Available: <https://fastapi.tiangolo.com>
- [6] Meta AI Research, FAISS, A Library for Efficient Similarity Search, GitHub Repository, 2021. Available: <https://github.com/facebookresearch/faiss>

[7] Anthropic, "Practical Guide to Building RAG Applications," Technical Blog, 2024. Available: <https://www.anthropic.com/research>

[8] React Team, React 18 Official Documentation. Available: <https://react.dev>